

JAVA DOCS

Generics in Java

Type Parameters, Bounded Types, Wildcards & Type Erasure

Java Agentic AI — Knowledge Base Reference

1. Why Generics?

Generics (Java 5) add compile-time type safety to classes, interfaces, and methods that operate on a parameterized type, eliminating manual casts and catching type errors at compile time instead of as a runtime `ClassCastException`.

```
// Before generics
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0);    // manual cast, runtime risk

// With generics
List<String> list2 = new ArrayList<>();
list2.add("hello");
String s2 = list2.get(0);           // no cast needed, compiler-enforced type
```

2. Generic Classes

```
public class Box<T> {
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}

Box<String> stringBox = new Box<>();
stringBox.set("hello");

Box<Integer> intBox = new Box<>();
intBox.set(42);

// Multiple type parameters
public class Pair<K, V> {
    private K key;
    private V value;
    public Pair(K key, V value) { this.key = key; this.value = value; }
    public K getKey() { return key; }
    public V getValue() { return value; }
}
```

3. Generic Methods

```
public static <T> void printArray(T[] array) {
    for (T item : array) System.out.println(item);
}

public static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}

Integer[] nums = {1, 2, 3};
printArray(nums);
System.out.println(max(5, 10));    // type inferred from arguments
```

4. Bounded Type Parameters

A bound restricts what types can be substituted for a type parameter, giving access to methods of the bound within the generic code.

```
public class NumberBox<T extends Number> {           // T must be Number or a subclass
    private T value;
    public double doubled() { return value.doubleValue() * 2; } // Number's method
    available
}

// Multiple bounds: one class (max one), plus any number of interfaces
public static <T extends Comparable<T> & Serializable> T max(T a, T b) { ... }
```

5. Wildcards

Wildcard	Syntax	Meaning	Use case
Unbounded	List<?>	Unknown type	You only need methods that don't depend on the type p
Upper-bounded	List<? extends Number>	Number or any subtype	Producer — you only READ elements as Number
Lower-bounded	List<? super Integer>	Integer or any supertype	Consumer — you only WRITE Integer or subtype elements

```
public static double sum(List<? extends Number> list) { // producer - safe to read
    double total = 0;
    for (Number n : list) total += n.doubleValue();
    // list.add(1); // COMPILE ERROR - can't safely add, actual type unknown
    return total;
}

public static void addNumbers(List<? super Integer> list) { // consumer - safe to write
    list.add(1);
    list.add(2);
    // Integer x = list.get(0); // COMPILE ERROR - can only safely read as Object
}
```

Note: PECS mnemonic: 'Producer Extends, Consumer Super' — use extends when the structure only produces (you read) values, use super when it only consumes (you write) values.

6. Type Erasure

Java generics are implemented via **type erasure**: generic type information exists only at compile time for type-checking purposes. At bytecode level, all generic type parameters are erased and replaced with their bound (Object if unbounded), with compiler-inserted casts added where needed. This preserves backward compatibility with pre-generics (Java 1.4 and earlier) bytecode.

```
List<String> strList = new ArrayList<>();
List<Integer> intList = new ArrayList<>();
System.out.println(strList.getClass() == intList.getClass()); // true - both are just
ArrayList at runtime

// Consequences of erasure:
```

```
// 1. Cannot use 'instanceof' with a parameterized type
// if (obj instanceof List<String>) { }           // COMPILE ERROR
if (obj instanceof List<?>) { }                   // OK - unbounded wildcard only

// 2. Cannot create a generic array directly
// T[] arr = new T[10];                           // COMPILE ERROR
T[] arr = (T[]) new Object[10];                  // unchecked cast workaround

// 3. Cannot overload methods that erase to the same signature
// void process(List<String> l) { }
// void process(List<Integer> l) { }               // COMPILE ERROR - both erase to process(List)
```

7. Generic Interfaces & Bounded Class Hierarchies

```
public interface Repository<T, ID> {
    T findById(ID id);
    void save(T entity);
}

public class UserRepository implements Repository<User, Long> {
    public User findById(Long id) { ... }
    public void save(User entity) { ... }
}
```

Note: This exact pattern (*Repository*) is the foundation of Spring Data JPA's *JpaRepository* — covered in [Spring_Data_JPA.pdf](#).

8. Quick Interview Recap

- What is type erasure and why does it exist? Generic type parameters are removed at compile time, replaced with their bound/Object; it exists to keep bytecode compatible with pre-Java-5 code and the JVM.
- What does PECS mean? Producer Extends, Consumer Super — a rule of thumb for choosing between ? extends T and ? super T wildcards.
- Why can't you create a generic array (new T[10])? Because at runtime the JVM has erased T and has no way to know what array type to actually allocate, risking type-unsafe `ArrayStoreException`.
- Can you overload methods differing only by generic type argument? No — after erasure both methods would have identical signatures, causing a compile error.
- What's a raw type? Using a generic class without its type parameter, e.g. `List` instead of `List` — allowed for backward compatibility but produces unchecked warnings and loses type safety.